COMPLETE PROJECT DOCUMENTATION

Book Scraper & Analytics Dashboard

An end-to-end Python application using FastAPI, SQLAlchemy, SQLite/PostgreSQL, and Books to Scrape

What is included

Source scraping ? Background scrape jobs ? Persistent book catalogue ? REST API ? Analytics dashboard ? Tests ? Migrations ? Render deployment guide

Project guide and runbook

Source: Books to Scrape practice catalogue ? Generated from the current project README

Contents

Features

Project structure

Run locally

How the project works

Push to GitHub

Deploy on Render

Limitations and future improvements

Book Scraper Assignment

A beginner-friendly web scraper and REST API that collects public book data from Books to Scrape, a website made for scraping practice. It stores results in SQLite for local development and supports PostgreSQL for persistent production storage.

Features

? Scrapes book titles, prices, availability, ratings, and product links.

? Follows the practice site's pagination and can collect its full 1,000-book catalogue in a background job.

? Saves books in the configured SQLite or PostgreSQL database and avoids duplicates using each product URL.

? Migrates existing rows from the legacy `books` table into the `products` table on startup without deleting the old table.

? Provides a FastAPI homepage and interactive API documentation.

? Includes a responsive, full-width Book Analytics dashboard with five summary cards, rating and price charts, highest-rated and lowest-priced book tables, and a refresh action.

? Provides a REST API for paginated book search, category/rating/price/availability filters, sorting, individual book lookup, and CSV export.

? Provides source scraping endpoints for recent books, the complete catalogue, and all pages of the Contemporary category.

? Tracks background scrape jobs and exposes job status and history endpoints.

? Stores book details and price observations in SQLite locally or PostgreSQL in production.

? Highlights highest-rated and lowest-priced books. The source does not publish sales counts, so the dashboard explicitly treats ratings as a popularity proxy rather than presenting them as verified best-seller data.

? Runs full-catalogue scrape jobs asynchronously and applies a process-local cooldown to reduce repeated public scrape requests.

Project structure

```
book-scraper-assignment/
├── app/
│   ├── __init__.py
│   ├── database.py
│   ├── jobs.py
│   ├── main.py
│   ├── models.py
│   ├── scheduled_scrape.py
│   ├── schemas.py
│   └── scraper.py
├── migrations/
├── data/
├── tests/
│   ├── test_api.py
│   ├── test_database.py
│   └── test_scraper.py
├── docs/
│   ├── Book_Scraper_Project_Report.md
│   └── screenshots/
│       ├── api-docs.png
│       └── dashboard.png
├── render.yaml
├── .gitignore
├── README.md
└── requirements.txt
```

Run locally

Requires Python 3.11.11 (or a compatible Python 3.11 release).

Windows PowerShell

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --reload
```

Run the automated test suite from the project directory:

```
python -m pytest -q
```

macOS/Linux

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --reload
```

Open http://127.0.0.1:8000 for the Book Analytics dashboard, http://127.0.0.1:8000/docs to scrape books and explore all API operations, or http://127.0.0.1:8000/health to check database connectivity. Unless `DATABASE_URL` is set, the SQLite database (`books.db`) is created automatically in the current working directory. It starts empty; run a scrape to populate it.

To use PostgreSQL locally, set `DATABASE_URL` to a PostgreSQL connection URL. The application accepts `postgres://` and `postgresql://` URLs and connects through the pinned psycopg driver.

How the project works

The application has four main steps:

1. **Scrape:** `app/scraper.py` requests the catalogue pages from Books to Scrape. BeautifulSoup finds each book card and extracts its title, GBP price, availability, rating, and product URL. A missing title, price, or URL makes that card invalid and it is skipped; missing availability or rating is shown as `Unknown`.
2. **Store:** `app/models.py` describes the `products` table. SQLAlchemy in `app/database.py` uses local `books.db` SQLite by default, or the database specified by `DATABASE_URL`. Configure PostgreSQL for Render so records are not kept on its temporary service filesystem. The product URL is unique so the same book cannot be stored twice. Existing rows from the older `books` table are copied during startup, while the original table is kept.
3. **Serve the API:** `app/main.py` provides routes for the dashboard, scraping, and reading/searching saved books. The API returns JSON for book records and scraping results.
4. **Display:** the homepage requests `/dashboard/stats` and `/api/analytics/overview` for its summary cards, charts, and ranked book highlights. Book listing, scraping, filtering, CSV exports, and scrape history are available through the REST API and Swagger UI.

Dashboard guide

? **Total books** is the number of records in the saved collection.

? **Average Rating** shows the mean of available one-to-five star ratings. The highest-rated-books table shows the top five titles and the rating, price, and availability for each.

? **Average price**, **Lowest price**, and **Highest price** summarize prices across the full collection in GBP.

? The rating and price charts summarize the saved collection.

? Use **Refresh Data** to reload the dashboard summaries and charts.

? Book browsing, filtering, CSV exports, scraping, and scrape history are available in Swagger UI at `/docs`.

? Books to Scrape has no sales figures, so actual sales-ranked best-sellers cannot be identified; the highest-rated-books panel is explicitly a ratings-based popularity proxy. The low-price panel displays the most affordable books.

Swagger UI (`/docs`) guide

Swagger UI is an interactive page generated by FastAPI from the API routes. Expand an operation to see its purpose and parameters. Select **Try it out**, fill in any values, and choose **Execute** to send the request and view its status and response.

Route	What it does	Parameters
`GET /`	Returns the dashboard HTML page.	None
`GET /health`	Checks API and configured database connectivity.	None
`GET /dashboard/stats`	Returns total books, top-rated titles, price summary, and availability counts for the full collection.	None
`POST /scrape`	Fetches books from the fixed source site and saves new records.	`limit`: defaults to `20`, minimum `1`, maximum `50`; the operation has a process-local 30-second cooldown.
`POST /api/scrape`	Queues a scrape job, including a full-catalogue scrape.	`limit`: defaults to `20`, minimum `1`, maximum `1000`.

Route	What it does	Parameters
`POST /api/scrape/contemporary`	Queues a scrape of the Books to Scrape Contemporary category across its pages.	`limit`: defaults to `1000`, minimum `1`, maximum `1000`.
`GET /api/scrape/status/{job_id}`	Returns status and result counts for a queued scrape.	Job ID.
`GET /api/scrape/history`	Lists recent scrape jobs.	Optional `limit` and `offset`.
`GET /api/analytics/overview`	Returns category, rating, and price-band chart data and book highlights.	None.
`GET /api/analytics/ratings`	Returns rating counts and average rating.	None.
`GET /api/analytics/prices`	Returns price summaries and book price histories.	Optional `limit` and `offset`.
`GET /api/analytics/availability`	Returns availability counts.	None.
`GET /products`	Returns a paginated product object.	`search`, `min_price`, `max_price`, `availability`, `category`, `rating`, `sort_by`, `sort_order`, `limit` (1–100), and `offset`.
`GET /products/export.csv`	Downloads matching products as `books.csv`.	`search`, `min_price`, `max_price`, `availability`, `category`, and `rating`.
`GET /products/{product_id}`	Returns one saved product.	Product ID in the path; missing IDs return `404`.
`GET /books`	Backward-compatible saved-book JSON array.	`search`, `min_price`, `max_price`, and `availability`; no pagination.

For example, `GET /products?search=light&min_price=10&max_price=60&availability=In%20stock` returns matching products within the inclusive price range. An invalid parameter (for example, `limit=0`) returns HTTP `422` with validation details. If the source site cannot be reached while scraping, the API returns HTTP `502`.

API endpoints

Method	Endpoint	Description
GET	/	Book Analytics dashboard
GET	/health	Checks API and database connectivity
GET	/dashboard/stats	Returns collection-wide total, rating, price, and stock summaries
POST	/scrape?limit=20	Synchronously collects up to 50 books; repeated calls are rate-limited
POST	/api/scrape?limit=1000	Queues a full-catalogue scrape (up to 1,000 books)
POST	/api/scrape/contemporary	Queues all books from the Contemporary category
GET	/api/scrape/status/{job_id}	Returns scrape-job status and counts
GET	/api/scrape/history	Lists past scrape jobs
GET	/api/analytics/overview	Returns category, rating, and price distributions and book highlights
GET	/api/analytics/ratings	Returns the rating distribution
GET	/api/analytics/prices	Returns current and historical price analysis
GET	/api/analytics/availability	Returns availability counts
GET	/products?limit=20&offset=0	Lists products with pagination and a total count
GET	/products?search=light&min_price=10&max_price=50&availability=In%20stock&category=Travel&rating=Five&sort_by=rating&sort_order=desc	Combines title, price, availability, category, rating, and sort options
GET	/products/export.csv?search=light	Downloads all matching books as a CSV attachment
GET	/products/{product_id}	Returns one product by database ID, or 404
GET	/books	Backward-compatible, unpaginated books list
GET	/books?search=light&min_price=10&max_price=50	Backward-compatible title and price filters

/products returns an object with total, limit, offset, and a products array. The legacy /books endpoint keeps returning a JSON array for existing clients. A synchronous scraping response reports products_collected, products_inserted, and products_skipped; background scraping returns a job_id, which can be polled at /api/scrape/status/{job_id}. Duplicate URLs are updated rather than duplicated. Invalid filter ranges return HTTP 422, a missing product returns 404, a source-site network failure returns 502, and a concurrent or too-frequent scrape returns 429.

End-to-end scrape and analytics flow

The dashboard is an analytics view; use Swagger UI (/docs) or the API below to collect and browse books:

1. Start the server and open /docs.
2. Execute POST /api/scrape/contemporary to collect the Contemporary category, or POST /api/scrape?limit=1000 to collect the full catalogue. The response includes a job_id.

3. Poll `GET /api/scrape/status/{job_id}` until the status is `completed`, `partial`, or `failed`.
4. View dashboard data at `/` or fetch chart data from `/api/analytics/overview`.
5. Browse saved books at `/products?category=Contemporary&limit=100`, inspect a book at `/products/{product_id}`, or export results from `/products/export.csv`.

From Windows PowerShell, this runs the Contemporary scrape and checks it through to saved results:

```
$job = Invoke-RestMethod -Method Post "http://127.0.0.1:8000/api/scrape/contemporary"
do {
    Start-Sleep -Seconds 2
    $status = Invoke-RestMethod "http://127.0.0.1:8000/api/scrape/status/$($job.job_id)"
} while ($status.status -in @("queued", "running"))
$status
Invoke-RestMethod "http://127.0.0.1:8000/products?category=Contemporary&limit=100"
Invoke-RestMethod "http://127.0.0.1:8000/api/analytics/overview"
```

Test each endpoint locally

In Swagger UI, expand each route and select **Try it out**. Or use a terminal:

```
curl http://127.0.0.1:8000/
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8000/dashboard/stats
curl http://127.0.0.1:8000/api/analytics/overview
curl -X POST "http://127.0.0.1:8000/scrape?limit=20"
curl -X POST "http://127.0.0.1:8000/api/scrape?limit=1000"
curl -X POST "http://127.0.0.1:8000/api/scrape/contemporary"
curl "http://127.0.0.1:8000/products?limit=10&offset=0"
curl "http://127.0.0.1:8000/products?search=light&min_price=10&max_price=50&availability=In%20stock"
curl -OJ "http://127.0.0.1:8000/products/export.csv?search=light"
curl http://127.0.0.1:8000/products/1
curl http://127.0.0.1:8000/books
curl "http://127.0.0.1:8000/books?search=light"
curl "http://127.0.0.1:8000/books?min_price=10&max_price=50"
```

The scrape route returns counts of collected, inserted, and skipped records. Repeating a scrape cannot add duplicate URLs. If the source website cannot be reached, the route responds with HTTP 502. Scraping only targets `books.toscrape.com`, observes a short delay between catalogue pages, and is throttled per application process.

Push to GitHub

This project is already connected to `https://github.com/bhumireddysivapriya/scapper.git`. Review and push changes from the project folder:

```
git add .
git commit -m "Improve book scraper API and report"
git push origin main
```

If using a different GitHub repository, create it first and set its URL as the `origin` remote. Do not initialize a new Git repository over this existing history.

Deploy on Render

1. Push the project to GitHub.
2. In Render, choose **New** → **Blueprint** to use `render.yaml`, or create a Web Service manually with these settings:

Setting	Value
Runtime	Python 3.11.11
Build command	`pip install -r requirements.txt`
Start command	`python -m alembic upgrade head && python -m uvicorn app.main:app --host 0.0.0.0 --port $PORT`
Instance type	Free

3. If using the Blueprint, it provisions a PostgreSQL database and supplies its connection URL to the web service. If you already have a Web Service, create or attach a Render PostgreSQL database and set the service's `DATABASE_URL` environment variable to its **Internal Database URL**. Do not commit the URL or credentials.
4. Deploy, then open `https://<your-service-name>.onrender.com/` for the dashboard or `/docs` for the API.

If the shell reports `gunicorn: command not found`, replace any `gunicorn` Start Command in the Render service settings with `python -m uvicorn app.main:app --host 0.0.0.0 --port $PORT`. Uvicorn is installed by `requirements.txt`; this project does not use Gunicorn.

Storage note: Render's service filesystem is ephemeral, so the default `sqlite:///./books.db` is for local development and can result in an empty dashboard after a production restart or redeploy. The app supports PostgreSQL through `DATABASE_URL`; configure a Render PostgreSQL database before scraping production records. Render's free database plan has its own lifecycle and storage limits, so check the current plan terms and arrange backups if the records must be retained long term. This repository change does not itself configure an existing Render service or verify a live deployment.

Limitations and future improvements

- ? Data is refreshed when someone calls a scrape endpoint or the configured scheduled scraper runs.
- ? The source does not expose sales metrics; sales-ranked best-seller lists cannot be calculated from its public catalogue.
- ? Production persistence depends on configuring a PostgreSQL database URL; the local SQLite default is not durable on Render.
- ? The process-local scrape cooldown is not shared across multiple server instances.